

Universidad de Puerto Rico

Recinto Universitario de Mayagüez



Capstone Final Report: Suffragium

Computer Science and Engineering

INSO4151

Dr. Marko Schutz

Diego L. I. Rodríguez Quintero (802-20-7831)

Ryan Williams Miranda (802-20-4536)

Eliel J. Hernández Vega (844-19-6970)

1. Introduction **2**
2. Problem Statement **3**
2.1 Problem definition 3
2.2 Target domain 4
2.3 Innovation 4
3. Project Objectives **5**
4. Solution Approach **5**
4.1 Gap in existing solutions 5
4.2 Value Proposition 6
4.3 Benefits and Limitations 6
4.4 Market Domain 7
4.5 Resources Needed 8
4.5 Intellectual Property 8
5. Technical Description **8**
5.1 Architecture/Framework to Build the Solution 8
5.2 Design Support Diagrams and Flowcharts 10
5.3 Related Works 14
5.4 Engineering Constraints 17
5.5 Engineering Standards 18
6. Project Plan **20**
7. References **22**
8. Preliminary Final Results **23**
8.1 Performance 23
8.2 Demonstration 29
9. Project Documentation **36**

1. Introduction

Voting system security, openness, and adaptability have been hot topics in the wake of last year's election in Puerto Rico and the United States. In light of news surfacing about sabotage, corruption and mismanagement of votes [1], numerous people have voiced their desire for a more reliable and strong voting process that guarantees all voices are fairly represented. Beyond political elections however, this problem also arises in other fields where decision-making is essential, like cooperative learning environments in the workplace and in academic organizations, whose leaderships require a democratic, and transparent, approach to elect.

One quick google search might offer solutions to these problems, providing different electronic voting options like *eBallot* [2] or *Simply Voting* [3], but a deeper dive into these platforms brings to light a lack of versatility between their domains. *Simply Voting* is just that, a simple, politically inclined, voting platform that physically mails its users' credentials,

while *eBallot* focuses on the electronic approach, programmatically assigning weights to administrator designed votes. However, they both suffer from administrative issues, requiring interviews with company representatives in order to configure methodologies and voter weights.

Suffragium is made for transparent and systematic decision-making processes where vote weight, accountability, and security tracking are crucial, in contrast to the simple polling tools seen in messaging apps like *WhatsApp*[20] or *Element*[19]. Our capstone project, Suffragium, aims to fill this vacant niche in the commercial market, offering a customizable, user-administered voting platform, where organizations of all kind can plan voting events, assign weights to users based on individualized authentication tokens, and not have to deal with an extensively complicated interface loop. Plan event, assign weights to users, vote, done.

2. Problem Statement

2.1 Problem definition

Addressing the problems introduced, what other circumstances impair the integrity of already available voting systems, and what groups could benefit from a solution to these issues? Group projects, where decision-making is crucial to the project's success, are one setting where an organized and open voting system might be especially helpful. Conventional group work frequently suffers from uneven involvement, a lack of accountability, and difficulties allocating duties equitably. Groups can guarantee that each member has a~~n equal~~ voice in the decision-making process by putting in place a structured voting system based on a weight-based allocation model.

Voters can assign a finite amount of points to various options under a weight-based voting system, ranking the options according to their perceived value or preference. This technique promotes deliberate decision-making, reduces bias, and offers a more democratic way to choose project directions, assign responsibilities, or assess contributions. A system like this might be created with flexibility in mind, enabling it to be tailored for a range of situations, including community decision-making, commercial collaborations, and class projects.

A clear and pertinent topic for a capstone project is creating a transparent and safe weight-based voting system for group projects. It entails addressing practical issues like security, fairness, and user involvement while including technological fixes like database administration, safe authentication, and an easy-to-use user interface. By tackling these

issues, the project could produce a useful instrument that promotes accountability in group decision-making, guarantees fair participation, and improves collaboration.

2.2 Target domain

Collaborative decision-making is essential to project success and operational effectiveness in academic, business, and organizational contexts. However, because they mostly use a one-person, one-vote method, current voting platforms and applications are not adaptable enough to support hierarchical voting structures. In situations when specific people like project managers, team leaders, or subject matter experts should have more control over choices due to their qualifications, duties, or leadership positions, this rigidity is problematic.Hence, this project aims to address this rigid precedent, offering a user-friendly, customizable voting experience for all types of organizations.

2.3 Innovation

Although there are many voting platforms and applications available today, their structures are still inflexible, with little flexibility in vote weighting or decision-making procedures. Situations where hierarchical voting structures are required are not accommodated by the majority of current systems, which function on a one-person, one-vote basis. This restriction is especially noticeable in collective decision-making, because some participants could require more control over the results because of their experience, leadership positions, or project duties.

Furthermore, there is no built-in method for settling disputes by a designated deciding vote on any of these platforms. Traditional systems require users to vote again in the event of a tie, which causes inefficiencies and delays in the decision-making process. This is especially troublesome in situations that call for prompt and decisive action, such group projects in school, business partnerships, and organizational decision-making. For instance, in a university software course, project teams can use Suffragium to democratically assign roles by vote weighting managers and team lead more heavily, allowing the leaders of the group to guide task assignments while preserving transparency.

We suggest creating a versatile weight-based voting system that enables users to give votes with varying weights according to predetermined standards in order to close these discrepancies. This system will incorporate a built-in deciding vote function to instantly resolve ties without the need for an extra voting round, and it will allow designated individuals to have varying degrees of voting influence. Our approach will improve decision-making processes in a variety of collaborative contexts by implementing these advances, which will increase justice, efficiency, and adaptability. The link will be given to the user upon the creation of the vote, making the process seamless and intuitive. A system like this might be created with flexibility in mind, enabling it to be tailored for a range of

situations, including community decision-making, commercial collaborations, and class projects.

3. Project Objectives

Develop a web-based voting platform by the end of the ~~2024~~ spring semester which can:

- Allow administrative users to create and schedule voting events. This includes:
  - Name
  - Start and end date
  - Description
  - Election Type (Public or Private)
  - Tie breaking methods
  - Candidates
  - Roles
- Assign user roles in order to allow or disallow participation in election events
- Designate weights based on election for each role
- Enable public or anonymous voting, tracked through administrative logs
- Configure the options for tiebreaking, including:
  - Random tiebreaker
  - Election administrator chooses
  - Repeating the vote
  - Role priority tie-breaker (User chosen role wins the tiebreaker)

4. Solution Approach

4.1 Gap in existing solutions

Even though there are several voting apps and platforms on the market today, they are devoid of crucial features that are necessary for effective dispute resolution in cooperative settings and hierarchical decision making. By implementing a weight-based voting system with weighted votes and an integrated deciding vote feature, our suggested solution closes these gaps. The main shortcomings of current solutions are listed below, along with how our initiative seeks to address them:

1. **Lack of Flexible Vote Weighing**
   - **Current Limitation:** Due to their one-person, one-vote paradigm, the majority of voting platforms currently in use are inappropriate for situations involving decision making where individuals ought to have more impact because of their experience, leadership, or project role.
   - **Our Solution:** Establish a weight-based system that allows users to assign vote weight according to preset standards, guaranteeing that votes accurately represent the levels of responsibility and skill.

2. **No Built-in Mechanism for Resolving Ties**
    - **Current Limitation:** An extra voting round is necessary in traditional voting systems in the event of a tie, which causes inefficiencies and delays in decision-making.
    - **Our Solution:** Put in place a deciding vote function that allows a designated person such as a senior management, professor, or team leader to automatically break ties without the need for a second voting round.
3. ~~**Limited Adaptability to Different Contexts**~~
    - ~~**Current Limitation:** Current voting methods are not customizable for group projects in school, business decision making, or organizational voting requirements; instead, they are made for general elections or simple surveys.~~
    - ~~**Our Solution:** Create a system that is adaptable and modular so that it can be customized for usage by corporate teams, businesses, community organizations, and educational institutions.~~
4. **Lack of Transparency and Accountability**
    - **Current Limitation:** While some systems such as *ElectionBuddy*[15] include audit logs, their depth and accessibility vary.
    - **Our Solution:** To guarantee that the decision making process is open and accessible to group members, include vote tracking, logs and reporting tools.

4.2 Value Proposition

~~Our project improves decision-making's justice, efficiency, transparency, and adaptability by implementing these cutting-edge characteristics. This solution offers an organized, effective, and user-centric method of group decision-making, guaranteeing improved results and more participation whether in educational institutions, business boardrooms, or community organizations.~~

Our platform is a cutting-edge, intuitive voting solution made for businesses, boardrooms, educational institutions or community organizations who need automated tie-breaking, weighted voting, and full audit transparency. Our system's user-friendly interface, integrated security, and thorough audit logs guarantee that each vote is given the weight it merits, facilitating effective, transparent, and reliable decision-making.

4.3 Benefits and Limitations

Suffragium offers native role-based weight control without the need for external setting, in contrast to *ElectionBuddy*[15]. It is easier to set up than tools that require manual vote aggregation or third-party plugins, but it is more complicated than one-click polling tools.

Client Benefits

1. **Improved Fairness and Representation**

- In order to guarantee that seasoned personnel have proportionate voice in decision making, clients can allocate varying voting weights according to leadership, role or expertise.

2. **Faster and More Efficient Decision Making**
    - In the event of a tie, the integrated decisive vote feature eliminates the need for unnecessary re-voting and avoids delays.
3. **Enhanced Transparency and Accountability**
    - A thorough audit trail keeps track of votes and guarantees that all decisions are documented and accessible for examination when necessary is advantageous to clients.
4. **Adaptability Across Different Use Cases**
    - With an emphasis on clarity and control over unrestricted customization, the system is made to be adaptable for particular organized use cases (such as board decisions and academic grading panels).

Limitations of the Solution

1. **Initial Learning Curve**
    - It could take some onboarding or training for users who are not accustomed with weighted voting to completely grasp how the system operates.
    - Guidelines or procedures regarding the equitable distribution of voting power among members may be required by organizations or stated before by groups.
2. **Potential Bias in Vote Weighting**
    - Assigning varying weights to votes may give the impression of unfairness or partiality if it is not set up correctly.
3. **Dependence on Digital Access**
    - Because the system requires a digital device and internet access, some users may not be able to utilize it in specific settings.
4. **Subjectivity in the Deciding Vote**
    - If not well controlled, the determining vote mechanism's reliance on a designated individual may create bias.
5. **Potential Resistance to Change**
    - Some institutions or organizations may be reluctant to use a new voting system because they are accustomed to using more conventional techniques.

4.4 Market Domain

The current commercial electronic voting market lacks the niche our project aims to fulfill. While there are plenty of options for structured, political, elaborate voting platforms, like *eBallot*, *Simply Voting,* or *Voatz*, which employs innovative biometric technologies, Suffragium aims to be a user friendly, customizable, simple experience for its clients, promoting a streamlined interface that even inexperienced voters can understand.

4.5 Resources Needed

In order to bring this project to fruition, the Suffragium team will employ the use of:

-Three laptops, in order to develop and share the programs necessary to create this web-based application
-Github, a proprietary developer platform which will allow us to create, store, manage, and share our code.
-React.js, Node.js, Next.js, Express.js, javascript runtime environments to enable development of our app
-To provide user authentication, data permanence, and scalability election processing, we will employ cloud services like *Supabase*[21] to deploy the backend. Hosting with permanent computation and storage layers will be necessary for logging and real-time result updates.

*Any further programming tools that forward the objectives of our project will be reported on the progress reports

4.5 Intellectual Property

All intellectual property generated by Suffragium (software, artwork and logos) will be open source. The Suffragium team reserves the right to change this at any moment. No individual member holds ownership of the project, but if any of them decides the advance the intellectual property beyond the scope of this project, they are free to do so, as long as they give notice to the other team members. Ownership of the property will belong entirely to the person who continues to pursue the project after the finish date. If two individuals choose to continue, the ownership will be divided equally between them. Similarly, if all three of us decide to move forward with the project, the shares will be split equally among us.

5. Technical Description

5.1 Architecture/Framework to Build the Solution

System Architecture:

The platform is designed using a three-tier architecture to ensure scalability, maintainability, and a clear separation of concerns. The application leverages modern web technologies, specifically React.js with Vite for the frontend and Supabase for the backend and database layers.

1. Frontend (User Interface Layer)

- **Framework & Tooling:** Built with React.js and bundled using Vite for fast development and optimized builds.
- **Responsiveness:** The UI is designed to be responsive and accessible across both desktop and mobile platforms using modern CSS techniques and component libraries.
- **Routing:** Client-side routing is managed using React Router, enabling seamless navigation without full page reloads.
- **Authentication Integration:** Communicates directly with Supabase for authentication and session management via Supabase's client libraries.
- **Real-time UI:** Utilizes Supabase's real-time subscriptions for live updates to election statuses, vote counts, or announcements without needing manual refreshes.

2. Backend (Application Logic Layer)

- **Backend-as-a-Service (BaaS):** The application logic is handled through Supabase Edge Functions, which replace the need for a traditional Express.js backend.
- **Custom Business Logic:**
    - **Weighted Voting Algorithm**: Implemented to compute vote outcomes based on user-customizable vote weights.
    - **Tie-breaking Mechanisms**: Custom logic will be triggered server-side to resolve ties using predefined strategies (e.g., random selection, re-voting).
    - **Election Configuration:** Edge functions and Supabase's Row-Level Security (RLS) policies allow for dynamically configurable election types and permissions. Handled by user.
- **APIs:** Secure serverless endpoints built using Supabase Functions enable data processing and validation without exposing business logic to the client.

3. Database (Storage Layer)

- **Database Engine:** Uses PostgreSQL, fully managed by Supabase.
- **Schema Design:**
- **Users Table**
    - Stores user profiles, including email, name, role, and status.
    - Integrated with Supabase Auth for authentication and user identity management.
    - Uses UUIDs as primary keys for consistency and security.
    - Contains timestamps for account creation and updates.
- **Userreference Table**
    - Stores internal user information
    - Tied to Supabase user auth table
    - Uses **role** and **type** to determine user access to elections
- **User_Role Table**
    - Stores user specific **roles** and the **weight** they have when voting in specific elections
    - Tied to **user** table to assign specific **roles**

- **Elections Table**
    - Captures metadata for each election, including title, description, start and end times, visibility, and status.
    - Linked to the **Admin** user who created it.
    - Includes a configuration field for flexible options (e.g., voting mode, limits).

- **Votes Table**
    - Stores actual vote selections.
    - Connected to a **User** and a **Candidate**, ensuring traceability and consistency.
    - Enforced by constraints to prevent duplicate voting within an election.

- **Candidates Table**
    - Contains candidate details such as name, profile, and associated election.
    - Enables dynamic candidate listing per election.

Security & RLS (Row-Level Security):

- Fine-grained RLS policies ensure that users only access data they are authorized to view or modify.
- Example: A voter can only see their votes; the election creator can only manage their elections.

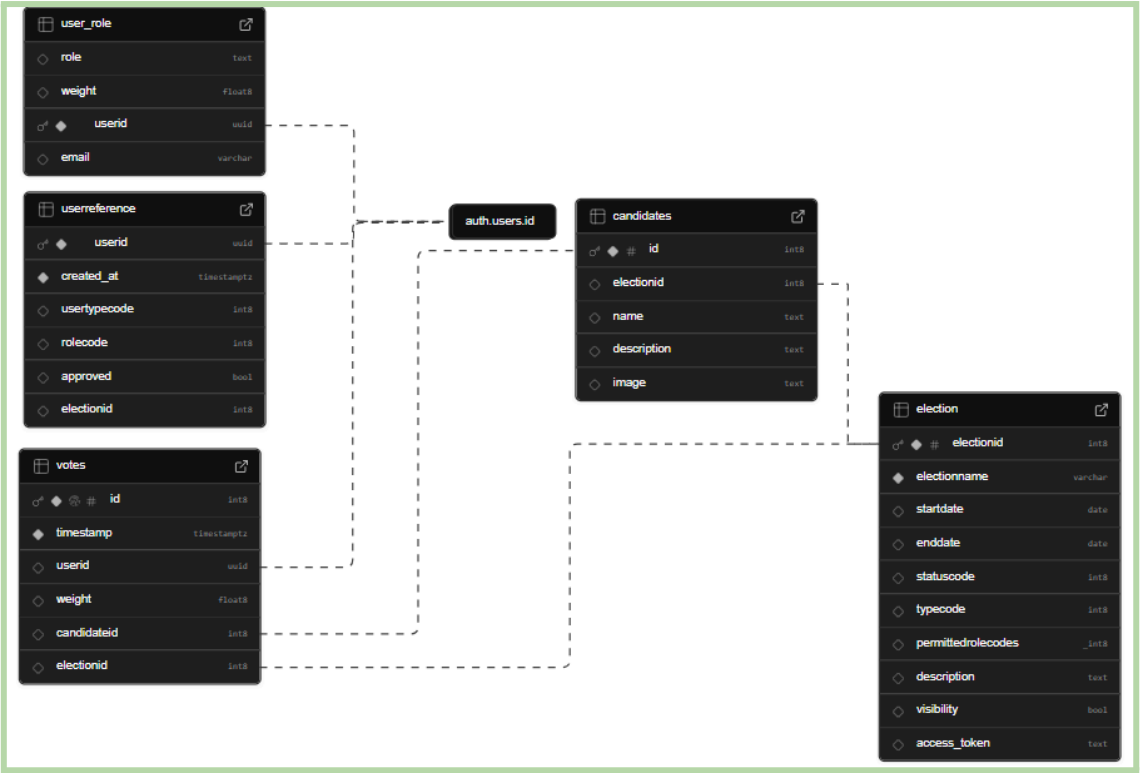
Real-time Subscriptions:

- Enabled via Supabase's real-time engine.
- Frontend clients subscribe to changes in the **votes** and **election** tables for immediate UI updates.

5.2 Design Support Diagrams and Flowcharts

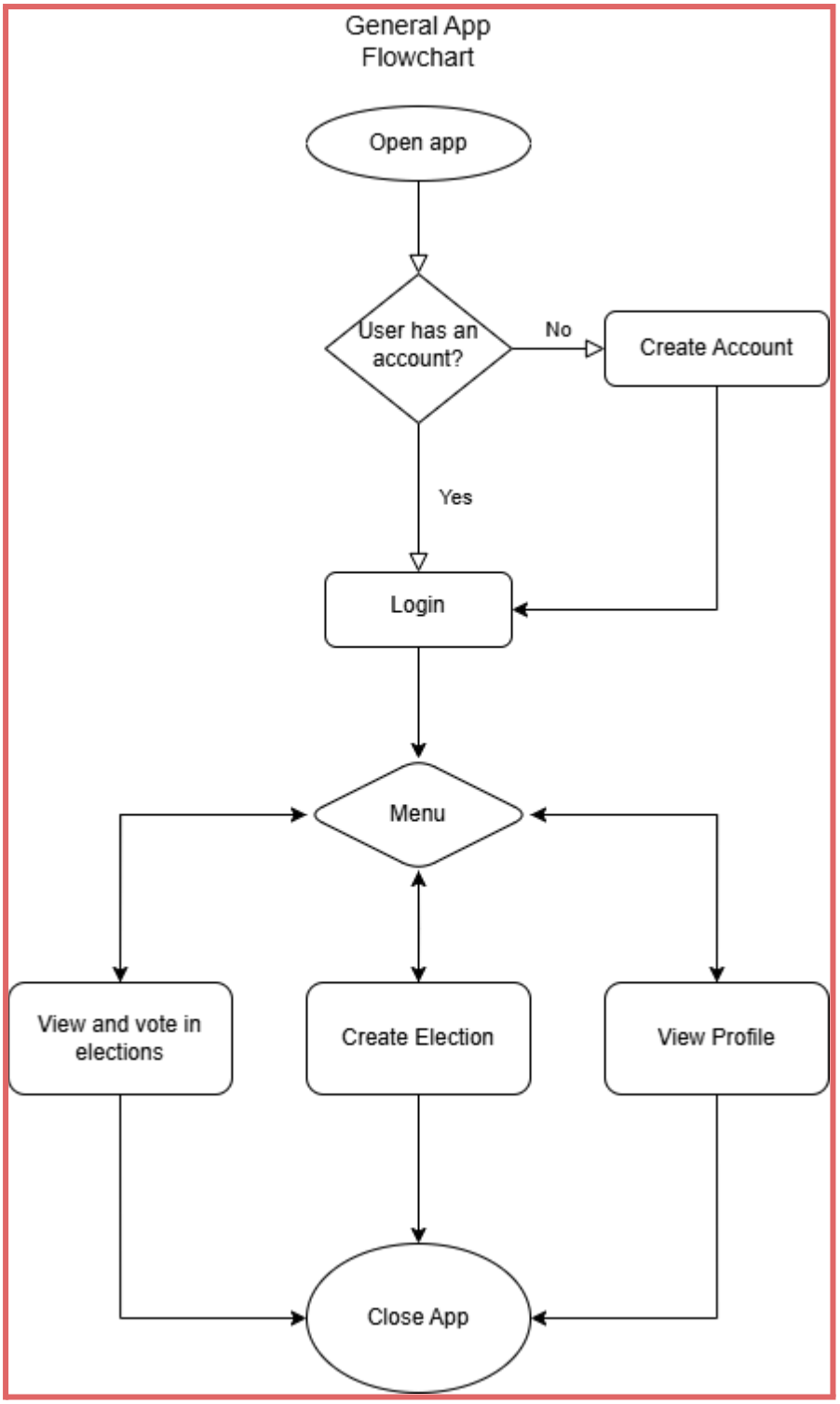
Entity-Relationship (ER) Diagram

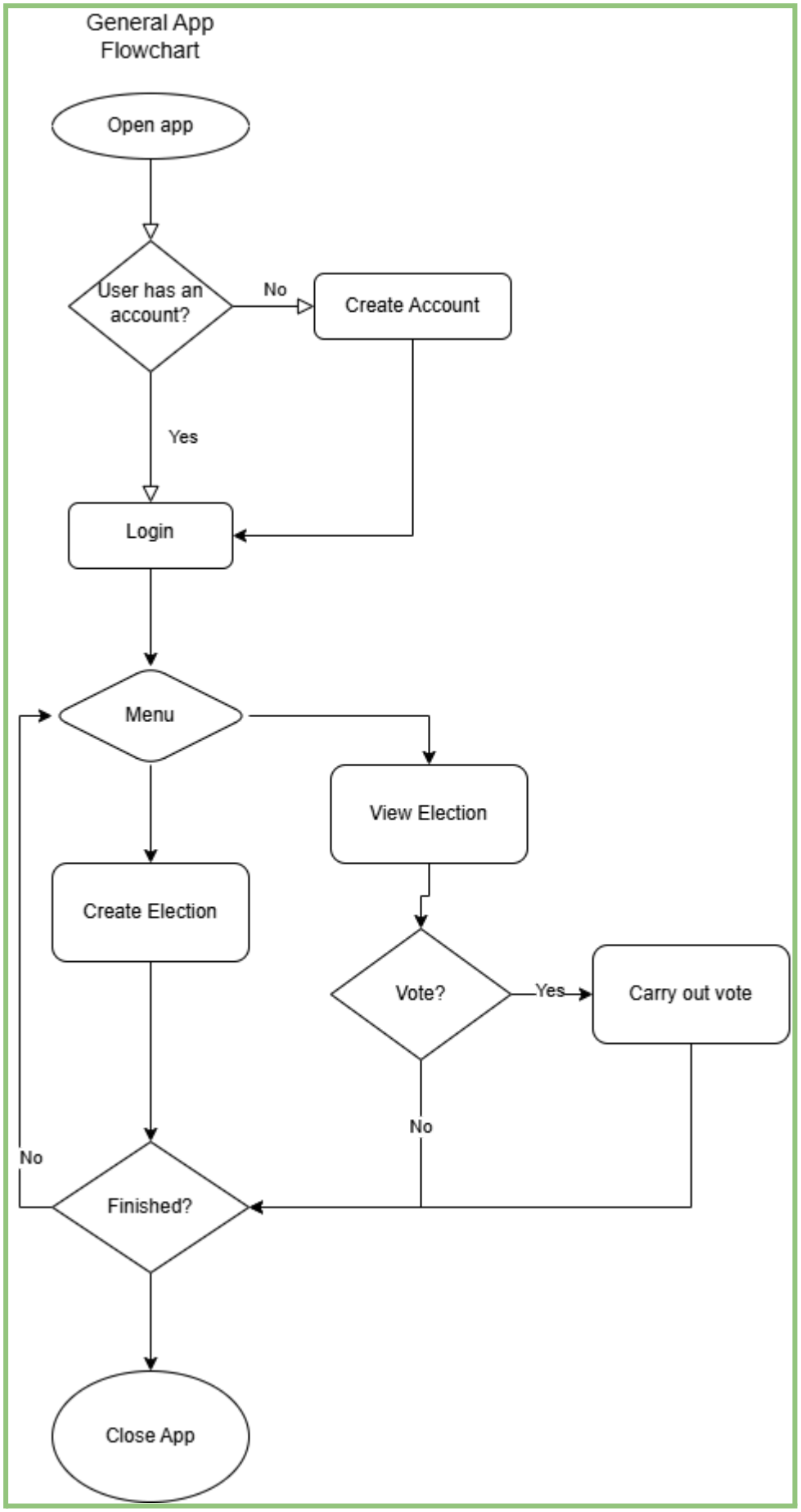
UPDATE ER DIAGRAM



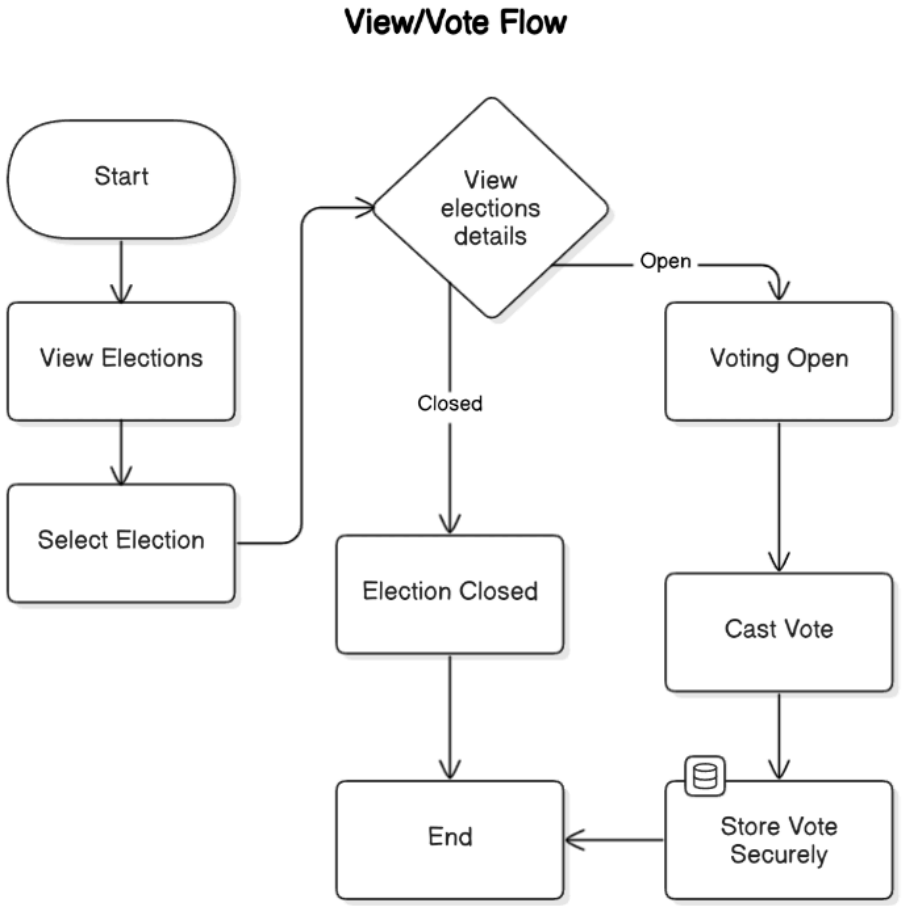
App development demanded a more comprehensive database in order to meet project requirements. Due to these changes, ER table diagrams were updated accordingly. Explanations for these changes are included in the Section 8.1 of this report.

System Flowcharts


General App Flowchart
Open app
User has an account?
No
Create Account
Yes
Login
Menu
View and vote in elections
Create Election
View Profile
Close App


General App Flowchart
Open app
User has an account?
No
Create Account
Yes
Login
Menu
View Election
Create Election
Vote?
Yes
Carry out vote
No
No
Finished?
Close App

1. General App Flow: Shows user flow and usage of the app. Due to the requirements of this flow, ER table diagrams needed to be updated in order to accommodate user capability to create elections.



2. Vote Submission Flow: The View/Vote Flow shows how a user goes through the voting process. It starts by viewing the list of elections and selecting one. If the election is open, the user can vote and their vote is stored securely. If it's closed, they're told voting isn't available. The process ends after voting or checking a closed election.

5.3 Related Works

Several online voting platforms exist, but they lack key customizations:

~~Existing Platform~~	~~Key Features~~
~~ElectionBuddy~~	~~• Provides email and SMS ballot distribution~~ ~~• Audit trails and real-time result updates~~ ~~• Supports a variety of election types~~

	● ~~Lacks direct user control over weight assignment.~~
~~OpaVote~~	● ~~Offers ranked-choice voting, approval voting and proportional representation~~ ● ~~Downloadable logs~~ ● ~~Doesn't support customizable vote weights or tie-breaking logic.~~
~~Voatz~~	● ~~Uses blockchain for tamper-proof voting records~~ ● ~~Biometric login and device authentication~~ ● ~~Primarily targeted at high-security institutional elections~~ ● ~~Its complexity and reliance on mobile biometrics limits accessibility.~~
~~Simply Voting~~	● ~~Has multiple election types~~ ● ~~Offers live results tracking~~ ● ~~Requires administrative setup through direct communication with their team.~~

Existing Platforms:

ElectionBuddy

The goal of ElectionBuddy, a cloud-based online voting platform, is to make elections safe, transparent, and easy to use for a variety of organizations, such as government agencies, associations, unions, nonprofits, educational institutions, and homeowner associations.

Key Features

- Allows for hybrid, in-person, and distant voting. In order to accommodate different participation preferences, voters can cast their ballots by mail, computer, tablet, or smartphone.
- Makes it possible to create ballots that include links, documents, candidate profiles, and images. supports a variety of voting methods, including approval, cumulative, ranked-choice, and first-past-the-post voting.
- Maintains election integrity by using 256-bit encryption, one-time voting keys, optional two-factor authentication, and voter anonymity.
- Maintains openness and confidence in the electoral process by offering verifiable audit trails and permitting independent third-party verification.
- Offers tools like automated reminders and real-time tracking of voter participation to increase turnout, and it makes it easier to notify voters by mail, SMS, or email.

Opavote

OpaVote is a user-friendly, reasonably priced, and safe online voting platform made to let businesses of all kinds hold democratic elections. In order to accommodate a variety of electoral requirements, it supports a large number of voting techniques, such as weighted voting, approval voting, and ranked-choice voting.

Key Features

- Organizations can select the voting method that best suits their needs with OpaVote's support for multiple voting systems, including Borda Count, Condorcet, Single Transferable Vote (STV), Instant Runoff Voting (IRV), and Approval Voting.
- Election officials have real-time access to data on voter participation, including email correspondence, ballot openings, and submissions. OpaVote offers comprehensive reports with interactive graphics after an election, ensuring results are transparent and easy to understand.
- Organizations with geographically scattered members would especially benefit from OpaVote's ability to seamlessly integrate on-site and remote voting with its support for both online and hybrid elections.

Voatz

Voatz is a mobile voting technology that uses web interfaces, tablets, and cellphones to make elections safe and accessible. It has been tested in a number of U.S. jurisdictions, such as West Virginia and Utah, and is mainly intended for voters with disabilities, abroad military members, and other populations that need remote voting options.

Key Features

- The platform uses a multi-step authentication procedure that includes using device biometrics like fingerprints or face recognition, taking live selfies, and scanning government-issued identification documents. To protect voter privacy, all personally identifiable information is removed after verification.
- A permissioned blockchain network is used to record votes, creating an audit trail that is unchangeable and impenetrable. A paper ballot is also produced for official tabulation at the jurisdiction for every vote.
- Voatz is accessible to voters with disabilities with features like Speech-to-text for write-in candidates, Voice Control on iOS, TalkBack on Android, and configurable font sizes.
- Voters receive anonymous, encrypted receipts to verify their choices after casting their ballots. These receipts improve openness and make post-election audits easier.

Simply Voting

A secure web-based platform called Simply Voting was created to let a variety of organizations, such as corporations, nonprofits, homeowner associations (HOAs), unions, and

colleges, conduct online elections. The platform has helped more than 5,000 organizations run more than 110,000 election events since its launch in 2003, totaling more than 17 million votes.

Key Features

- Includes features for candidate withdrawal, manual vote entry for paper ballots, voter segmentation, and weighted voting.
- Makes use of one vote per voter, encrypted ballots, and unique voter authentication. Time stamps and IP addresses are recorded for every action taken by administrators and voters.
- Supports a number of voting methods, including cumulative, ranked-choice, approval, and first-past-the-post.

What Sets the Platform Apart?

There are some key features that sets us apart from other platforms, which are mainly focused in making the platform customizable. The platform will include multiple voting methods which include majority, ranked-choice, proportional. The features that sets us apart will be the following:

1. Weighted voting: which allows votes to carry different weights based on predefined roles.
2. Automatic tie-breaking: pre-assigned deciding votes resolve election ties.
3. Enhanced transparency: Audit logs, voter receipts, and real-time verification will be implemented.

We looked at frameworks for Multi-Criteria choice Making (MCDM), which prioritize comparative choice evaluation. One such framework is the Analytic Hierarchy Process (AHP). Although these techniques offer deep decision modeling, they frequently call for intricate computations and specialized knowledge. Instead, Suffragium emphasizes the usability and real-world implementation of weighted voting in regular teamwork situations.

5.4 Engineering Constraints

~~Several constraints must be addressed to ensure the platform's security, privacy, and usability:~~

There are many engineering limitations on our voting platform that have an immediate influence on implementation plans and design choices. These consist of compliance, usability, scalability, privacy, security, and transparency.

1. **Privacy & Data Protection**

- Personally Identifiable Information (PII) such as names and email addresses will be stored.
- ~~Anonymized vote storage to protect voter identities.~~
- Compliance with data protection laws.

2. Security Considerations

- Role-Based Authentication to prevent unauthorized access.

3. Scalability & Performance

- Optimized database indexing and caching to handle voters efficiently.
- Load balancing & server redundancy to ensure uptime.

4. Usability & Accessibility

- Mobile-Friendly UI to enable voting from smartphones.

5. Transparency & Auditability

- Tamper-Proof Logging to track all vote transactions.
- Publicly Verifiable Election Logs for audit transparency.

5.5 Engineering Standards

Our approach will be guided by multiple established engineering standards to ensure security, accessibility, and reliability. While we may not strictly adhere to one specific standard, we will integrate best practices from several relevant frameworks to achieve a well-balanced and secure system. Some of the key standards we will consider include:

1. **Security & Data Protection**

What it is:

- The ISO/IEC(Information Security Management) 27001 standard, which is widely accepted for information security management, will be used. It offers a methodical way to handle private user and business data, guaranteeing its availability, confidentiality, and integrity. Establishing, implementing, maintaining, and continuously improving an information security management system (ISMS) are all covered by the standard. Risk assessment, security controls, data access policies, and incident response are all covered by its rules and procedures.

How it will be used:

- Put encryption in place to store user information and votes.
- Implement policies for access control (e.g., role-based access in the admin dashboard).
- To identify and address threats, use secure logging and auditing procedures.

2. **Software & Web Development**

What it is:

- IEEE 1012(Software Verification & Validation), a comprehensive standard that outlines the procedures for making sure a software system is both correctly constructed (verification) and satisfies the requirements and expectations of its users (validation), will be used. The standard offers instructions for organizing, carrying out, and recording testing procedures at every phase of the software development lifecycle. To make sure the program works as intended in practical settings, it places a strong emphasis on traceability, test preparation, error detection, and quality assurance.

How it will be used:

- Test the voting logic and backend thoroughly at the unit, integration, and system levels.
- Verify usability during test voting rounds with actual users.
- To monitor feature coverage, use formal test plans and traceability matrices.

3. **Voting System Standards**

What it is:

- The VVSG(Voluntary Voting System Guidelines) standard, a collection of optional technical guidelines published by the U.S. Election Assistance Commission (EAC), will be used by us to direct the creation of safe, dependable, and easily accessible voting systems. These rules address usability, auditability, accessibility for those with disabilities, voter privacy, and system integrity. Their purpose is to guarantee that voting platforms uphold public confidence, facilitate precise vote tallying, and safeguard the electoral process's security.

How it will be used:

- Make the user interface accessible by including features like keyboard navigation and screen reader assistance.
- Make sure voters and administrators can audit and validate every vote.
- Provide thorough election logs and real-time outcome checking.

What it is:

- The CIS Controls(Center for Internet Security) guidelines, a collection of 18 top cybersecurity practices created by professionals to assist businesses in safeguarding their systems and data against frequent online attacks, will be used by us. These controls include suggestions for virus defense, data security, monitoring, incident response, secure configuration, system hardening, and access control. Many sectors have embraced the CIS Controls as a viable and scalable means of enhancing cybersecurity posture.

  How it will be used:

- Use secure configurations to harden servers and databases.
- Turn on logging and receive alerts for any unusual activity.
- Make use of strong password policies and multi-factor authentication.

6. Project Plan

6.1 Activities and steps that lead to completion of the project

Phase 1: Research & Planning (Weeks 1-3, August - February 14)

- Research existing voting systems, security measures, and usability requirements.
- Identify legal considerations related to online elections.
- Define the system architecture, database structure, and primary voting methods.

Deliverables:

- Preliminary proposal submission

Phase 2: System Design & Prototyping (Weeks 4-6, February 15 - March 14)

- Refine the ER diagram, flowcharts and create wireframes.
- Set up the development environment and version control (GitHub/GitLab).
- Develop a basic frontend and backend prototype to test API communication.
- Implement authentication and role-based access control.

Phase 3: Core Feature Development (Weeks 7-11, March 15 - April 14)

- Build the frontend UI for voters and administrators.
- Implement backend logic for elections, voting, and security measures.
- Integrate vote validation, and database storage.
- Implement key features, including:
    - Weighted voting
    - Tie-breaking mechanism

    - Real-time results display
    - Audit logs for transparency

Deliverables:

- Functional election creation and voting system
- Working backend

Phase 4: Testing, Refinement, and presentation (Weeks 12-14, April 15 - May 12)

- Collect user feedback from test participants.
- Optimize performance and scalability.
- Refine database queries and caching strategies.
- Prepare the final presentation and demonstration.

Deliverables:

- Final Documentation
- Project Presentation

6.2 Milestone & Project Deliverables

Milestone	**Estimated Completion Date**	**Deliverables**
Project Proposal Submission	February 14, 2025	Project proposal
Prototype Development	March 17, 2025	Initial system architecture, Backend prototype, Front-end UI prototype
Progress Report Submission	April 14, 2025	Updated design, Front-end UI, Backend logic, Database setup initial testing
Final Report Submission & Demo and Presentation	May 12, 2025	Final documentation, refined system, test results

6.3 Team Roles & Responsibilities

Team Member	**Primary Responsibilities**	**Additional Responsibilities**
Eliel J. Hernández Vega	Frontend	Database, Documentation

Diego L. I. Rodríguez Quintero	Backend	Database, Documentation
Ryan Williams Miranda	Backend	Database, Documentation

7. References

1. W. Maldonado, “Amplio esquema de robo de votos corrompe el sistema electoral en Puerto Rico,” *Centro De Periodismo Investigativo*. Available: https://periodismoinvestigativo.com/2024/09/robo-votos-corrompe-elecciones-puerto-rico/
2. *EBallot | secure online voting platform & election software*. (n.d.). https://www.eballot.com/
3. Lack, B. (2024, December 13). *Full managed election - simply voting*. Simply Voting. https://www.simplyvoting.com/managed-elections/
4. Aid. (2024, February 29). Navigating constraints in software development: balancing quality and efficiency. Apriorit. https://www.apriorit.com/dev-blog/how-to-manage-constraints-in-software-development-effectively
5. Bahr, N. E. (2024, November 13). How to write a software development proposal? Nandbox Native App Builder. https://nandbox.com/software-development-proposal-how-to-write/
6. CIS Critical Security Controls. (n.d.). CIS. https://www.cisecurity.org/controls
7. Eraser – Diagram design. (n.d.). https://app.eraser.io/dashboard/all
8. How to write a successful software development proposal | UpTech. (2024, May 20). https://www.uptech.team/blog/software-development-proposal
9. IEEE Standards For Software Verification and Validation. (n.d.).
10. ISO - Standards. (n.d.). ISO. https://www.iso.org/standards.html
11. Nazarbayev University LibGuides: Engineering Standards: About Standards. (n.d.). https://nu.kz.libguides.com/standards_guide
12. React. (n.d.). https://react.dev/
13. Serna, M. (2024, March 29). A guide to writing a technical approach. Key Solutions Blog. https://info.ksiadvantage.com/blog/a-guide-to-writing-a-technical-approach
14. Voluntary Voting System Guidelines | U.S. Election Assistance Commission. (n.d.). https://www.eac.gov/voting-equipment/voluntary-voting-system-guidelines
15. ElectionBuddy. (2023, October 30). Voting Online, Simplified with ElectionBuddy - ElectionBuddy. https://electionbuddy.com/
16. Lack, B. (2025, January 28). Online voting tool - Simply Voting. Simply Voting. https://www.simplyvoting.com/
17. Online voting and elections website | OpaVote. (n.d.). https://opavote.com/
18. Voatz. (2020, September 9). Home - Voatz secure and convenient voting anywhere. https://voatz.com/
19. “Element | Secure collaboration and messaging.” https://element.io/

20. WhatsApp, “WhatsApp,” *WhatsApp.com*. https://www.whatsapp.com/
21. “Supabase | the open source Firebase alternative,” *Supabase*. https://supabase.com/

8. ~~Preliminary~~ Final Results

8.1 Performance

Updated Requirements and Functionality Overview (4/14/2025)
New entity-relationship requirements surfaced throughout app development, necessitating modifications to the original architecture. To better fit project objectives and semester time limits, the following adjustments were made:

- **User-Centric Election Creation**: There is no longer a requirement for distinct administrator or configurator roles because users can now directly create and configure elections. This streamlines procedures, empowers users, and makes development easier.
- **Vote Weight Configuration**: Vote weighting based on roles is now managed during election creation. At this point, users designate roles and allocate weights, guaranteeing that the election maker maintains configuration control.
- **Email-Based Role Assignment**: Role assignments employ user emails as identifiers. Assigning weighted voting rights is made easier using this solution, which removes the need for users to actively look for and identify other users.

A new Entity-Relationship (ER) diagram and user flowchart have been created to accommodate these modifications. This report's Section 5.2 contains these.
Throughout the development of Suffragium, the application evolved significantly beyond the original proposal. Several new features and critical fixes were implemented based on iterative testing and user feedback. The most important accomplishments are listed below:

Completed Features

The Suffragium voting platform has successfully included the following functionalities. These features meet the platform's fundamental needs for usability, security, transparency, and flexibility.

Voting System & Election Mechanics

- Weighted Voting System

    - Vote weights are allocated to users according to responsibilities that the election creator specifies. Election results are directly impacted by vote weight, which enables modifiable power dynamics in the process of making decisions.
- Tie Breaking Mechanisms
  Multiple strategies are supported for resolving election ties:
    - Random winner selection
    - Manual winner assignment by the election creator
    - Role-based tie-breaking (e.g., higher-privileged roles decide)
    - Revote among tied candidates using a new election instance
- Manual Winner Assignment
    - After voting concludes, creators can choose the winner in elections set up with manual tie-breaking.

Access Control and Security

- Role Based Access Controls
    - Roles assigned via email determine a user's access to elections. Certain elections can only be viewed or participated in by authorized users.
- Private Election Token System
    - Access tokens generated by private elections are unique and can only be accessed via a specific URL. Voting and joining require tokens.
- Public Direction Directory
    - By default, public elections are displayed and available without tokens. This preserves control over private processes while guaranteeing discoverability.
- Revote Protection
    - Voters are not allowed to cast more than one ballot in a single election. Following the initial submission, vote submissions are logged and locked.

Transparency and Auditability

- Voting Audit Logs and Trails
    - Timestamped vote logs and receipts are available to voters, promoting transparency and verifiability without disclosing the substance of votes.
- Election Status and Display
    - In both the management and viewing interfaces, elections are marked as "Open" or "Ended" with clarity.
- Voting Results UI
    - Current vote totals, top contenders, and the ultimate victor are displayed in real-time, with tie-breaking logic applied when necessary.

User Interface and Experience

- Candidate Image Upload
    - Candidates may submit profile pictures, however there are size restrictions. Previewed images are presented uniformly in all perspectives.
- Always Visible Candidate List

    - Clarity and accessibility are increased because candidate information is always visible without the need for user toggles.
- Tabbed Election Layout
    - Using dynamic rendering for improved navigation and filtering, elections are arranged into "Available" and "Ended" tabs.
- Responsive Design
    - Every page is mobile and desktop device optimized. Consistent appearance and a simple, minimalist layout characterize the interface.
- UI Improvements and Bug Fixes
    - Buttons are disabled during submission to prevent duplicate elections.
    - Navigation and dropdown issues resolved.
    - Time formatting errors corrected.
    - Confirmation email delivery bug (Safari-specific) fixed.

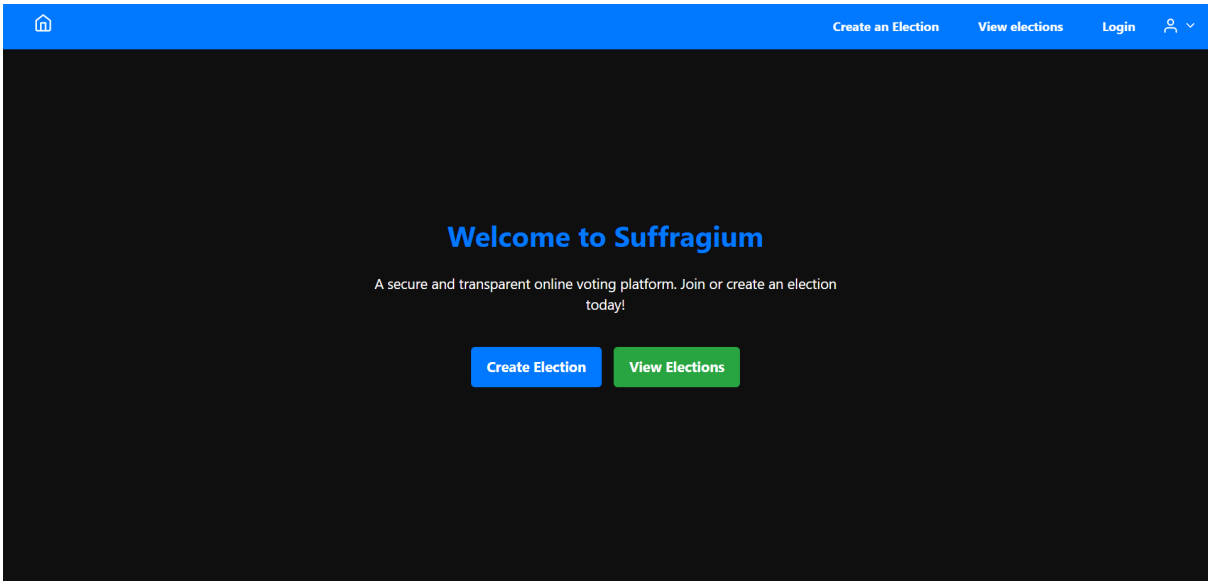
Authentication and Deployment

- OAuth Integration
    - For a smooth and safe login process, users can use Supabase Auth to sign in with Google or GitHub.
- Session Handling and Routing
    - Users are properly redirected (e.g., to the election creation page) after logging in. Restricted pages cannot be accessed by unauthorized users.
- Github Pages Deployment
    - With a custom domain and HTTPS enforced for security, the live site is hosted at suffragium.online.
- User Navigation Menu
    - User profile, settings, and logout options are easily accessible through a dropdown menu in the navbar.

- Voting Link Visibility Management
    - To avoid misunderstanding and needless exposure in public listings, tokenized access links are only shown for private elections.

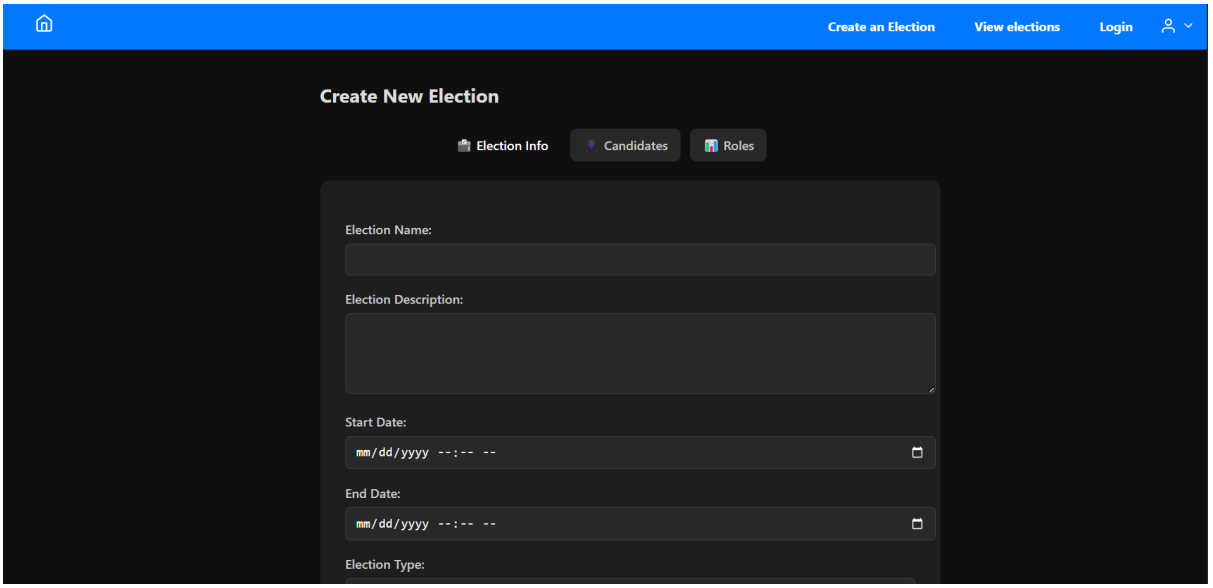
Front-End Features and Page Functionality

Welcome Page



The Welcome page provides a brief description of the page and lets the user navigate the functions of the web page with buttons and a navigation bar.

Create Election Page



The election creation interface is divided into three tabs:

1. **Election Info**
2. **Candidates**
3. **Roles**

Election Info Tab:

- **Public/Private Election Option:**
    - Public: The election is visible and open to all users.

    - Private: An access_token (URL) is generated. Only users with this URL can view and join the election.

- **Election Type Selection:**
    - *Normal*
    - *Weighted*

- When a private election is selected, the generated URL is displayed with instructions to share it with intended voters.

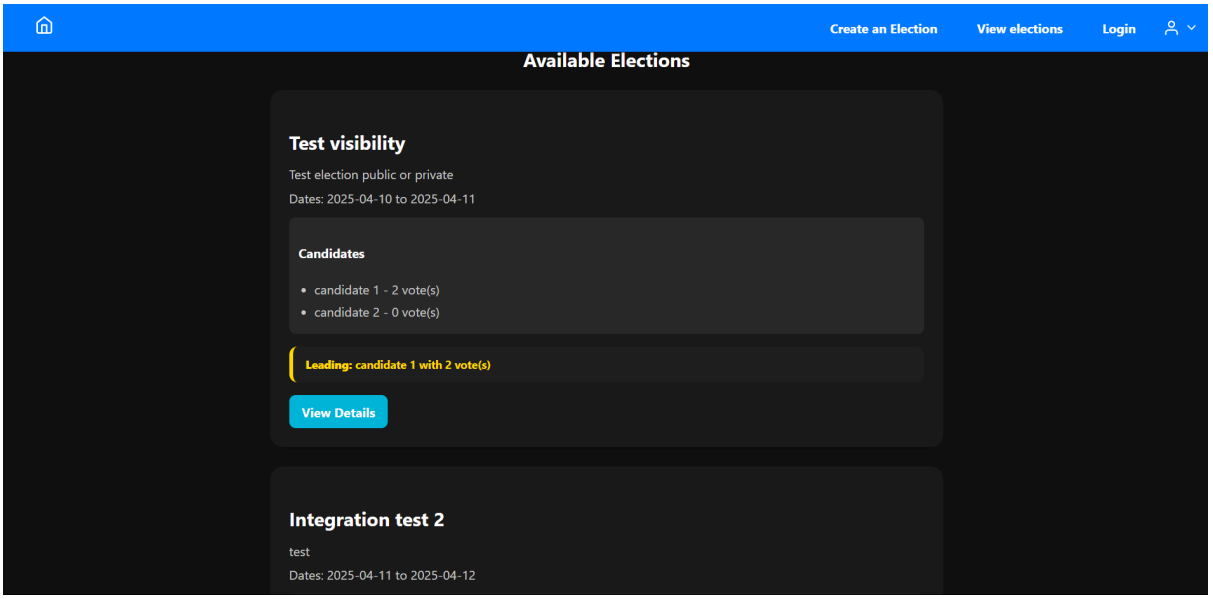
Candidates Tab:

- Users can add candidates.
- Option to upload an image for each candidate.

Roles Tab:

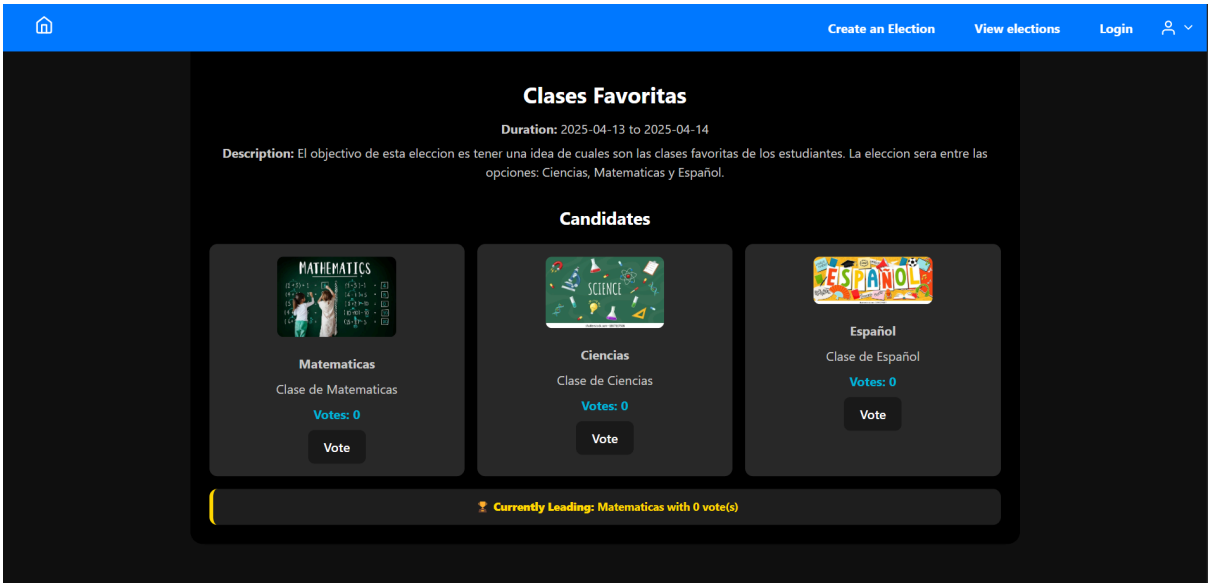
- Users can define roles and assign a weight to each.
- Roles can be assigned to voter emails manually.

View Elections Page



The View Election page displays a list of *public* elections only, and it shows the **leading candidate** for each election.

- Each election entry includes a **"View Details"** button:
    - Clicking reveals full candidate details (including images if available).
    - A **"Join Election"** button appears for users to participate.
    - Upon joining, users are presented with voting options.



This is a Sample election interface where students vote for their favorite subject. Each card represents a candidate with an image, description, and vote option.

Testing Summary

Throughout development, thorough testing was conducted when a feature or a new bug fix was implemented to ensure feature stability and correct database integration. No automated tests were made, these are the tests we ran on our website to ensure website stability:

~~**General Testing**~~

- ~~Verified successful upload of all components to the database.~~
- ~~Confirmed proper front-end behavior after code changes.~~
- ~~Performed integration testing after each new feature was implemented.~~
- ~~Validated login functionality.~~

~~**Election Creation Testing**~~

- ~~Attempted to create an election without candidates (confirmation message prompts for at least two candidates).~~
- ~~Tested both *normal* and *weighted* election types.~~
- ~~Verified functionality of public and private election settings.~~
- ~~Confirmed candidate data (including images) is correctly displayed.~~
- ~~Verified correct creation and assignment of roles by email.~~

~~**View Election Testing**~~

- ~~Ensured only public elections appear on the main view page.~~
- ~~Verified correct and dynamic display of available elections and candidate data.~~
- ~~Confirmed backend data is accurately read and rendered.~~
- ~~Attempted to join an election without being logged in (correctly prevented).~~

- ~~Verified entire voting process, including:~~
  - ~~Role-based voting participation.~~
  - ~~Accurate summation of role weights into final vote tally.~~

General Testing

- Data Integration Testing: Verified successful upload and synchronization of all components to the Supabase backend.
- Regression Testing: Confirmed consistent front-end behavior after code modifications.
- Continuous Integration (CI) Testing: Performed integration testing after each new feature was implemented to ensure end-to-end stability.
- Authentication Validation: Validated login functionality against expected authentication workflows.

Election Creation Testing

- Negative Testing: Attempted to create an election without candidates; verified that appropriate validation messages enforce business rules.
- Functional Testing: Tested both normal (equal vote) and weighted election types for expected behavior.
- Configuration Testing: Verified that both public and private election settings were correctly saved and applied.
- Data Rendering Validation: Confirmed accurate display of candidate information, including associated images.
- Role Assignment Testing: Verified correct creation and assignment of user roles via email notification and backend linking.

View Election Testing

- Negative Testing: Attempted to create an election without candidates; verified that appropriate validation messages enforce business rules.
- Functional Testing: Tested both normal (equal vote) and weighted election types for expected behavior.
- Configuration Testing: Verified that both public and private election settings were correctly saved and applied.
- Data Rendering Validation: Confirmed accurate display of candidate information, including associated images.
- Role Assignment Testing: Verified correct creation and assignment of user roles via email notification and backend linking.

8.2 Demonstration

The final project is deployed and accessible through GitHub Pages with a custom domain:

- Live Project: https://suffragium.online
- GitHub Repository: https://github.com/Elielh17/Suffragium
- Demo Link: https://youtu.be/MY0JJ3PkoOc

This demonstration section provides visual and technical evidence of the platform's capabilities. Screenshots, code excerpts, and interface walkthroughs are included to illustrate how Suffragium delivers advanced election functionality through TypeScript logic and Supabase integration.

Key demonstration highlights include:

- Creating both public and private elections.
- Adding candidates with descriptions and profile images.
- Assigning vote weights by role and verifying weighted vote tallies.
- Showcasing tie-breaking scenarios and resolution logic (including revotes).
- Viewing election status, audit logs, and winner announcements in real time.
- Full authentication, role assignment, and participation flow.

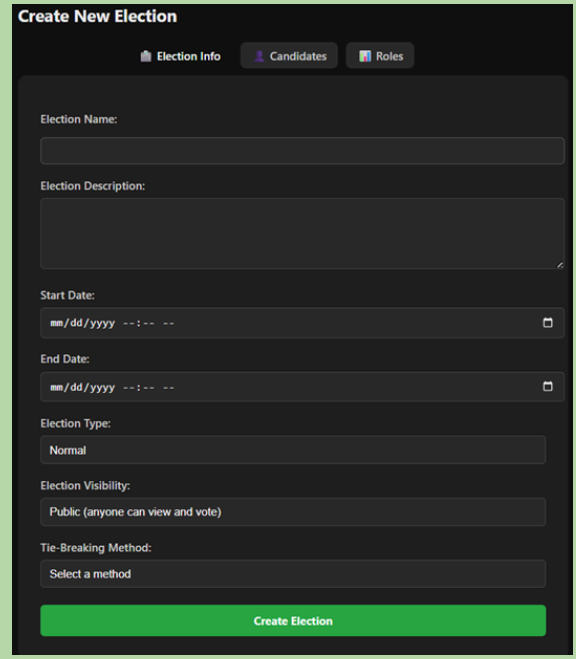
The following key TypeScript functions form the backbone of the voting platform's logic and user interactions:

Create Election page:

1. handleSubmit – Creates a new election

This is the core submission handler that:

- Validates election setup (requires at least 2 candidates and roles if weighted)
- Creates a new election in the Supabase election table
- Uploads candidates and their optional images
- Assigns user roles and vote weights via the user_role table
- Redirects the user to the Manage Election page on success




```
const handleSubmit = async (e) => {
  e.preventDefault();
  if (isSubmitting) return;
  setIsSubmitting(true);

  if (typeCode === 2 && roles.length === 0) {
    alert("Please add at least one role for a weighted election.");
    setIsSubmitting(false);
    return;
  }

  if (candidates.length < 2) {
    alert("Please add at least two candidates to create an election.");
    setIsSubmitting(false);
    return;
  }

  const formattedStartDate = startDate;
  const formattedEndDate = endDate;
  const isPrivate = visibility === "private";
  const accessToken = isPrivate ? uuidv4() : null;

  const {
    data: { session }
  } = await supabase.auth.getSession();

  const electionData = {
    electionname: electionName,
    description: electionDescription,
    startdate: formattedStartDate,
    enddate: formattedEndDate,
    statuscode: 1,
    typecode: typeCode,
    permittedrolecodes: [],
    visibility: visibility === "public",
    access_token: accessToken,
    userid: session.user.id,
    tie_break_type_id: tieBreakTypeId || null,
    tiebreak_role: tieBreakTypeId === 104 ? tiebreakRole : null
  };
```

```
const { data, error } = await supabase.from("election").insert([electionData]).select();
if (error || !data?.length) {
  alert("Failed to create the election.");
  setIsSubmitting(false);
  return;
}
const electionId = data[0].electionid;

await supabase.from("candidates").insert(
  candidates.map((c) => ({
    electionid: electionId,
    name: c.name,
    description: c.description || null,
    image: c.image_url || null
  }))
);

await Promise.all(emailAssignments.map(async (assignment) => {
  await supabase.from("user_role").insert({
    email: assignment.email,
    role: assignment.role,
    weight: assignment.weight,
    electionid: electionId
  });
}));

const url = isPrivate
  ? `${window.location.origin}/Suffragium/#/view-elections?token=${accessToken}`
  : `${window.location.origin}/Suffragium/#/view-elections?id=${electionId}`;

setShareableLink(url);
setShareMessage("Election and candidates created successfully!");
setShowPopup(true);

navigate(`/manage-election?id=${electionId}`);
};
```

2. Candidate and Role Management Functions

These helper functions allow users to build the election setup before submitting:

- uploadImage - Uploads a selected candidate image to Supabase Storage. Returns a publicly accessible URL used in the UI and stored in the database.
- addCandidate - Adds a new candidate entry with name, description, and optional image. Internally calls uploadImage if an image file is selected.
- addRole - Adds a voting role with a custom label (e.g., “Professor”) and numeric vote weight. Roles are used only in weighted elections to determine vote power.
- removeCandidate - Removes a candidate from the list by index. Allows the election creator to revise the ballot before final submission.

```
const uploadImage = async (file) => {
  if (file.size / 1024 / 1024 > MAX_FILE_SIZE_MB) {
    alert(`Image exceeds ${MAX_FILE_SIZE_MB}MB size limit.`);
    return null;
  }
  const fileName = `${Date.now()}_${file.name}`;
  const { (alias) const supabase: SupabaseClient<any, "public", any>
  if (err import supabase
  return supabase.storage.from("candidate-pictures").getPublicUrl(fileName).data.publicUrl;
};

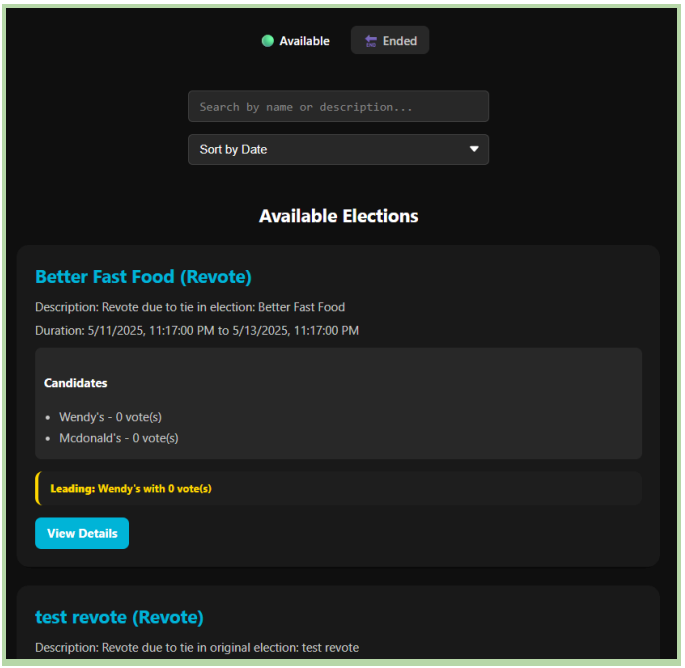
const addCandidate = async () => {
  if (!newCandidate.trim()) return;
  let imageUrl = null;
  if (newCandidateImage) {
    imageUrl = await uploadImage(newCandidateImage);
    if (!imageUrl) return;
  }
  setCandidates([...candidates, {
    name: newCandidate,
    description: newCandidateDescription.trim() || null,
    image_url: imageUrl || null
  }]);
  setNewCandidate("");
  setNewCandidateDescription("");
  setNewCandidateImage(null);
  setImagePreview(null);
};

const addRole = () => {
  if (!newRole.description.trim()) return;
  const newRoleWithId = { ...newRole, id: roles.length + 1 };
  setRoles([...roles, newRoleWithId]);
  setNewRole({ description: "", voteweight: 1 });
};

const removeCandidate = (index) => {
  setCandidates(candidates.filter((_, i) => i !== index));
};
```

View Election Page:

1. fetchData – Loads and Processes Election Data



```
const fetchData = async () => {
  const {
    data: { session },
  } = await supabase.auth.getSession();

  if (session?.user) {
    setUserId(session.user.id);
    setUserEmail(session.user.email);
    await fetchUserVotes(session.user.id);
    await fetchUserJoins(session.user.id);
  }

  const params = new URLSearchParams(location.search);
  const accessToken = params.get("token");
  const electionIdParam = params.get("id");

  let query = supabase
    .from("filteredelectionbydate")
    .select(`*, candidates(*, votes(*))`);

  if (accessToken) {
    query = query.eq("access_token", accessToken).eq("visibility", false);
  } else if (electionIdParam) {
    query = query.eq("electionid", electionIdParam);
  } else {
    query = query.eq("visibility", true);
  }

  const { data: electionsData, error } = await query;
  if (error) return console.error("Error fetching elections:", error);

  const electionIds = electionsData.map((e) => e.electionid);
  const validElectionIds = electionIds.filter(id => typeof id === "string" || typeof id === "number");

  const { data: roleAssignments, error: roleError } = await supabase
    .from("user_role")
    .select("email, weight, electionid")
    .in("electionid", validElectionIds);

  if (roleError || !roleAssignments) {
    console.error("Failed to fetch user roles:", roleError);
    return;
  }
```

```
const emailWeightMap = {};
for (const r of roleAssignments) {
  emailWeightMap[`${r.electionid}|${r.email}`] = r.weight;
}

const processedElections = electionsData.map((filteredelectionbydate) => {
  const candidates = filteredelectionbydate.candidates.map((candidate) => {
    const voteCount = candidate.votes?.reduce((sum, vote) => {
      let weight = 1;
      if (filteredelectionbydate.typecode === 2 && vote.email) {
        const key = `${filteredelectionbydate.electionid}|${vote.email}`;
        weight = emailWeightMap[key] || 1;
      }
      return sum + weight;
    }, 0);

    return {
      ...candidate,
      voteCount,
    };
  });

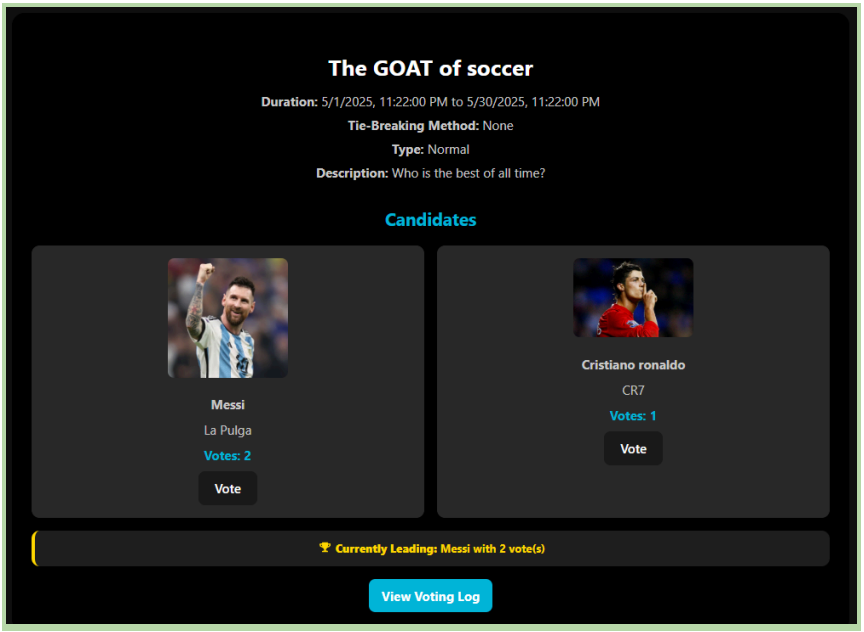
  const sortedCandidates = [...candidates].sort((a, b) => b.voteCount - a.voteCount);

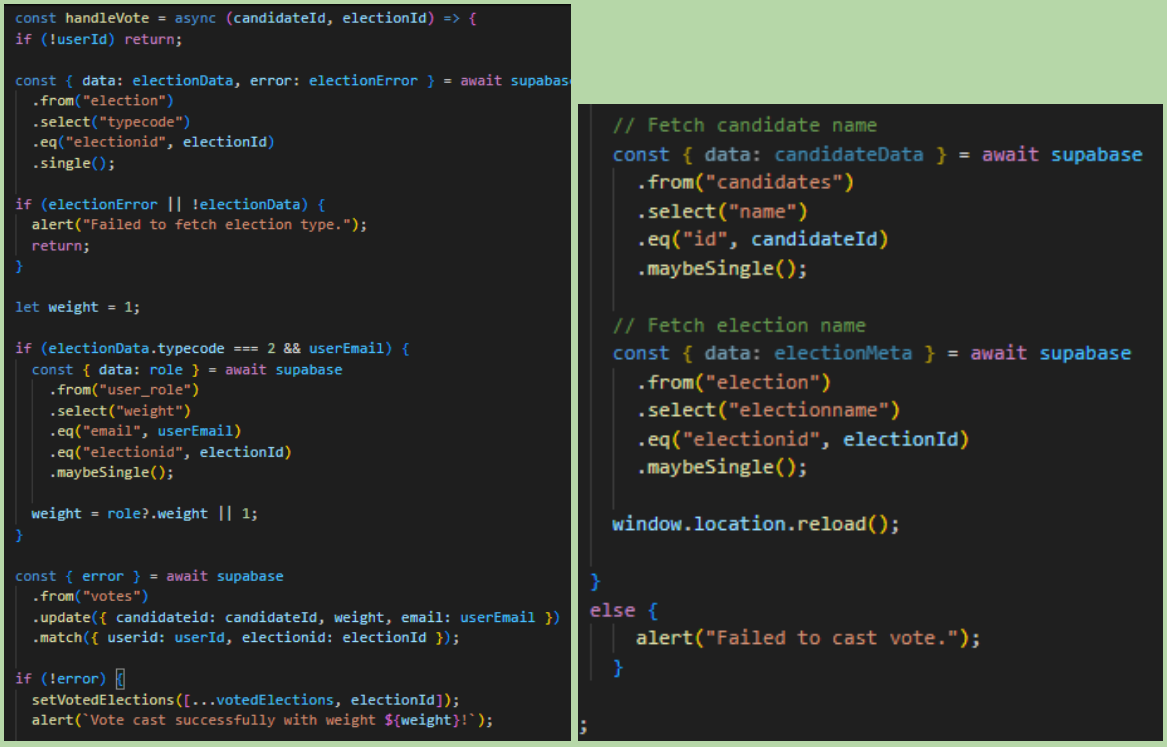
  const topVotes = sortedCandidates[0]?.voteCount || 0;
  const tiedCandidates = sortedCandidates.filter(c => c.voteCount === topVotes);
  const isTie = tiedCandidates.length > 1;

  return {
    ...filteredelectionbydate,
    candidates: sortedCandidates,
    winner: sortedCandidates[0] || null,
    isTie,
    tiedCandidates: isTie ? tiedCandidates : []
  };
});

setElections(processedElections);
if (processedElections.length === 1 && (accessToken || electionIdParam)) {
  setSelectedElection(processedElections[0]);
}
setLoading(false);
};
```

2. handleVote – Casts a vote





```
const handleVote = async (candidateId, electionId) => {
if (!userId) return;

const { data: electionData, error: electionError } = await supabas
  .from("election")
  .select("typecode")
  .eq("electionid", electionId)
  .single();

if (electionError || !electionData) {
  alert("Failed to fetch election type.");
  return;
}

let weight = 1;

if (electionData.typecode === 2 && userEmail) {
  const { data: role } = await supabase
    .from("user_role")
    .select("weight")
    .eq("email", userEmail)
    .eq("electionid", electionId)
    .maybeSingle();

  weight = role?.weight || 1;
}

const { error } = await supabase
  .from("votes")
  .update({ candidateid: candidateId, weight, email: userEmail })
  .match({ userid: userId, electionid: electionId });

if (!error) {
  setVotedElections([...votedElections, electionId]);
  alert(`Vote cast successfully with weight ${weight}!`);

  // Fetch candidate name
  const { data: candidateData } = await supabase
    .from("candidates")
    .select("name")
    .eq("id", candidateId)
    .maybeSingle();

  // Fetch election name
  const { data: electionMeta } = await supabase
    .from("election")
    .select("electionname")
    .eq("electionid", electionId)
    .maybeSingle();

  window.location.reload();

}
else {
    alert("Failed to cast vote.");
  }

;
```

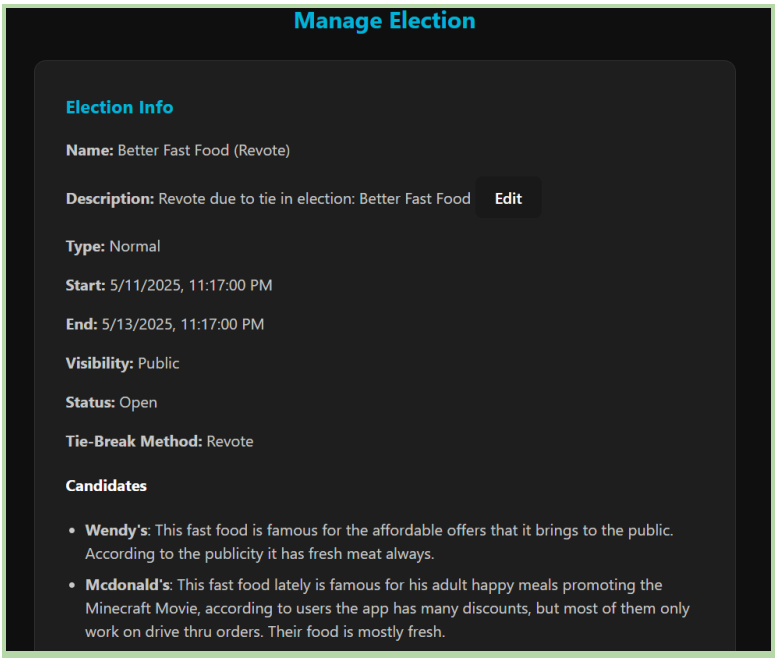
My Elections/Manage elections:

My Elections / Manage Election Page:

1. loadElection – Loads and verifies election ownership

This function retrieves the election using either an access token or ID, then:

- Verifies that the logged-in user is the creator of the election
- Loads associated candidates from the `candidates` table
- Fetches all votes and builds a vote count map
- Detects tied candidates if the election has ended and matches the tie-break method
- Automatically loads tied candidates and shows the winner selection UI if “Creator Decides” or “Revote” is configured




```
const loadElection = async () => {
  const { data: tieData } = await supabase.from("types").select("id, description").gte("id", 101);
  if (tieData) setTieBreakTypes(tieData);

  const {
    data: { session },
  } = await supabase.auth.getSession();
  if (!session) return navigate("/login");
  setUser(session.user);

  let query;
  if (accessToken) {
    query = supabase.from("election").select("*, candidates(*)").eq("access_token", accessToken);
  } else if (electionId) {
    query = supabase.from("election").select("*, candidates(*)").eq("electionid", electionId);
  } else {
    return alert("No election identifier provided.");
  }

  const { data, error } = await query;
  if (error || !data) return alert("Election not found.");
  const electionData = Array.isArray(data) ? data[0] : data;
  if (electionData.userid && electionData.userid !== session.user.id) {
    alert("You are not authorized to manage this election.");
    return navigate("/");
  }
  setElection(electionData);
  setNewDescription(electionData.description);

  const { data: candidates } = await supabase
    .from("candidates")
    .select("*")
    .eq("electionid", electionData.electionid);

  if (candidates) {
    electionData.candidates = candidates;
    setElection({ ...electionData, candidates });
  }
```

```
  const { data: votes } = await supabase
    .from("votes")
    .select("candidateid, email")
    .eq("electionid", electionData.electionid);

  const voteCountMap = {};
  for (const vote of votes) {
    if (!vote.candidateid) continue;
    voteCountMap[vote.candidateid] = (voteCountMap[vote.candidateid] || 0) + 1;
  }

  const maxVotes = Math.max(...Object.values(voteCountMap));
  const tied = Object.entries(voteCountMap).filter(([_, count]) => count === maxVotes);

  if (electionData.tie_break_type_id === 105 && new Date(electionData.enddate) < new Date() && tied.length > 1) {
    setTiedCandidates(candidates.filter(c => tied.map(([id]) => parseInt(id)).includes(c.id)));
    setShowWinnerPopup(true);
  }
};
```

2. createElectionFromTemplate – Clones election for revote

Triggered when a revote is needed, this function:

- Copies the configuration of the original election
- Only includes candidates that were tied
- Sets a new start/end date for the revote
- Inserts a new election and candidates into the database
- Returns a shareable link for participants

```
const createElectionFromTemplate = async (templateElection, candidateList, startDate, endDate) => {
const {
  data: { session }
} = await supabase.auth.getSession();

const accessToken = templateElection.visibility ? null : crypto.randomUUID();

const electionData = {
  electionname: `${templateElection.electionname} (Revote)`,
  description: `Revote due to tie in election: ${templateElection.electionname}`,
  startdate: startDate,
  enddate: endDate,
  statuscode: 1,
  typecode: templateElection.typecode,
  permittedrolecodes: templateElection.permittedrolecodes || [],
  visibility: templateElection.visibility,
  access_token: accessToken,
  userid: session.user.id,
  tie_break_type_id: templateElection.tie_break_type_id,
  tiebreak_role: templateElection.tiebreak_role || null
};
```

9. Project Documentation

Project code repository

- ~~https://github.com/Elielh17/Suffragium.git~~
- https://github.com/Elielh17/Suffragium

Project web

- ~~https://elielh17.github.io/Suffragium/#/~~
- https://suffragium.online

Elevator pitch

- https://www.youtube.com/watch?v=98rfld-KNv4

Presentation slides

- https://docs.google.com/presentation/d/1Zh9Jx8UvnBRZkP9t1m4_V_O6XyJTBYx_/edit?usp=sharing&ouid=112039750489046807030&rtpof=true&sd=true